Comprehensions and iteration

Module 2 - Lesson 3

Overview

Comprehensions construct lists, dicts, and sets from iterables in a single readable expression. The iterator protocol powers loops, generator expressions, and lazy processing.

Key Concepts

- A list comprehension is [expr for item in iterable if condition].
- Dict comprehensions build key/value pairs: {k: v for ...}.
- Set comprehensions deduplicate automatically.
- Generator expressions use parentheses and yield items lazily, saving memory.
- The built-in functions zip, map, filter, sorted, and enumerate compose with iteration.
- Iterators are single-use; lists are reusable; generators produce values on demand.

Example

```
nums = range(1, 11)
squares = [n * n for n in nums if n % 2 == 0]
print(squares)  # [4, 16, 36, 64, 100]

pairs = {k: v for k, v in zip("abc", [1, 2, 3])}
# {'a': 1, 'b': 2, 'c': 3}
```

Summary

Comprehensions express transforms in one readable line, and generators process large data lazily. Together they replace most loops with clear, efficient expressions.

Practice Checklist

- Rewrite a for-loop that builds a list as a comprehension.
- Build a dict of counts using a dict comprehension.
- Use a generator expression inside sum() to add even squares lazily.